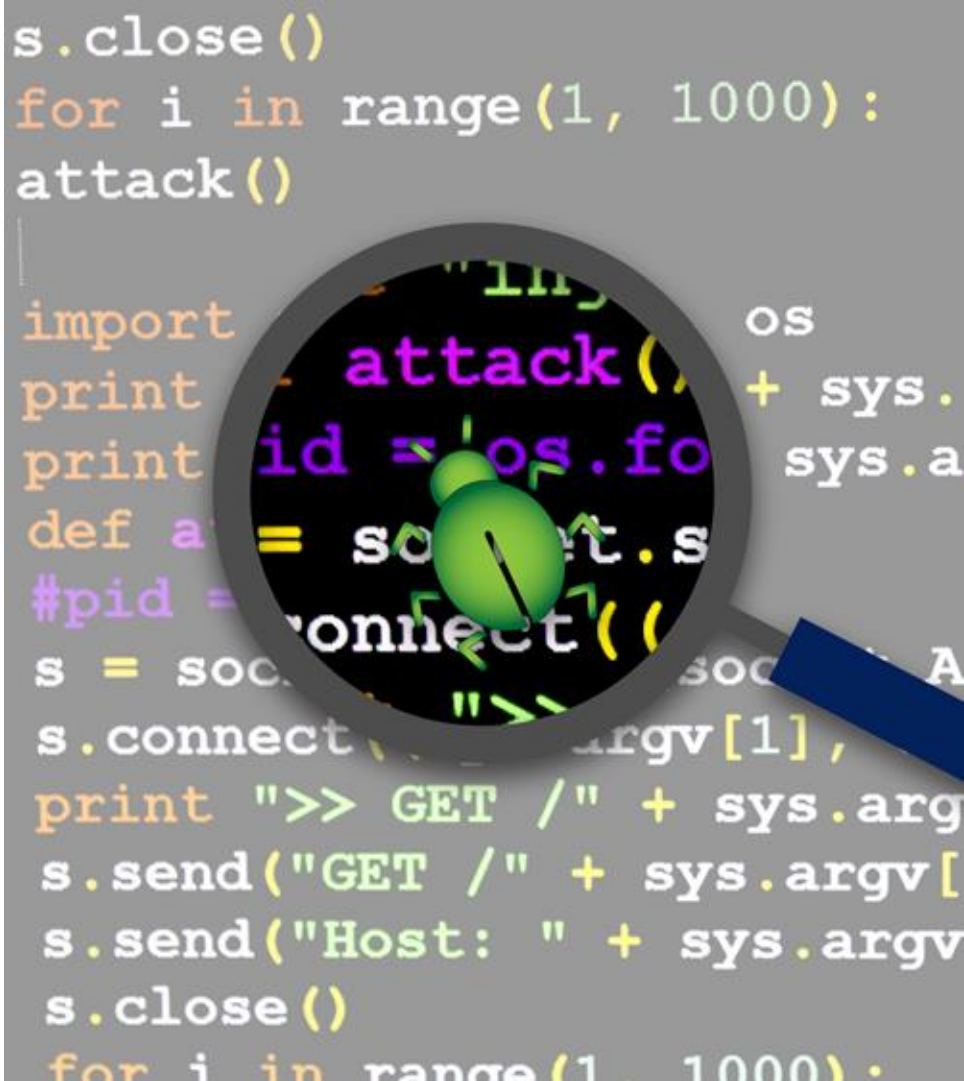
CIS 5470
Software Analysis

Introduction to Software Analysis

Mayur Naik

Penn Engineering



Outline

- Course Logistics
- Course Overview & Introduction
- Lab 1 Recitation
- Introduction to Software Analysis
- Tradeoffs in Software Analysis

Course Logistics

Course Staff

Instructor

Mayur Naik

AGH 642B

mhnaik@cis.upenn.edu

Teaching Assistants

Claire Wang

AGH 642

cdwang@cis.upenn.edu

Zain Aamer

AGH 642

zaamer@cis.upenn.edu

Course Material

- Lecture videos and transcripts: https://cis547.github.io/lectures/
- **No required textbook** - All materials provided
- *Recommended:* Static Program Analysis free at https://cs.au.dk/~amoeller/spa/

- Course draws material from many books

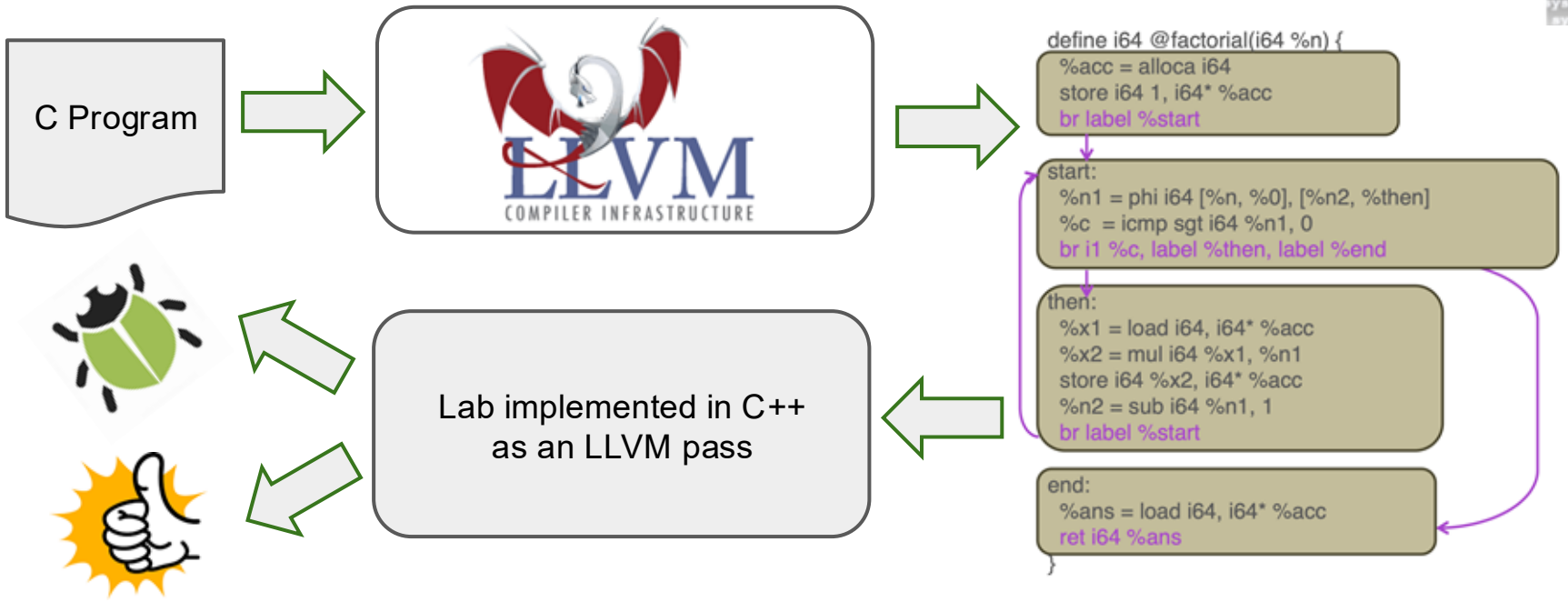
Communication

- From students:
  - Post to Ed
  - During or after class
- From the staff:
  - Announcements on Ed
- In person:
  - Meet Instructor or TA by appointment
  - Set appointment by email

What to Expect

- Roughly one software analysis topic covered in class each week
- Roughly one lab (programming assignment) corresponding to the topic each week except Fuzzing and Dataflow which get 2 weeks each
  - Each lab typically due on Wednesday midnight; Lab 1 due on Sept 2
- Labs must be done individually
- Discussion of technical and implementation concepts of labs is encouraged (e.g. on Ed) but you must not copy solutions

Architecture of Each Lab



Grading (Tentative)

Type		%	Description
Concept Comprehension Quizzes		3%	Each week has video quizzes providing an opportunity to review the concepts you have learnt.
Labs	There are 8 programming assignments called Labs (Lab 1 - Lab 8). Labs will be graded using Gradescope. You need to access Gradescope through the external tool on Canvas.		
	Labs 1 through 8	54%	Labs 1, 2 (overview) are worth 4% and 6% resp. Labs 3, 4, 5 (dynamic analysis) are worth 8%, 7%, and 7% resp. Labs 6, 7, 8 (static analysis) are worth 8%, 7%, and 7% resp.
Group Project		23%	Project will be done in teams of 2-3 students.
Final Exam		20%	There is one cumulative final exam in person.

Prereqs

- Programming experience: ability to write code in a language like C, Java, or Python. 6 Labs use C/C++ and 2 use Python.
- Data structures and algorithms: trees, graphs, search algorithms, and asymptotic analysis.

CIS 1200 / CIT 5940

- Systems programming: ability to install, run, and read scripts. — CIS 2400 / CIT 5950
- Mathematical foundations: set theory, logic, probability, and formal proofs. — CIS 1600 / CIT 5920

If Labs 1 and 2 interest you, you will be able to pick up even if you aren't fluent in some of the above topics.

Course Overview and Introduction

Why Take This Course?

- Learn modern methods for improving software quality
  - reliability, security, performance, etc.
- Become a better software developer/tester
- Build specialized tools for software diagnosis and testing
- For the war stories

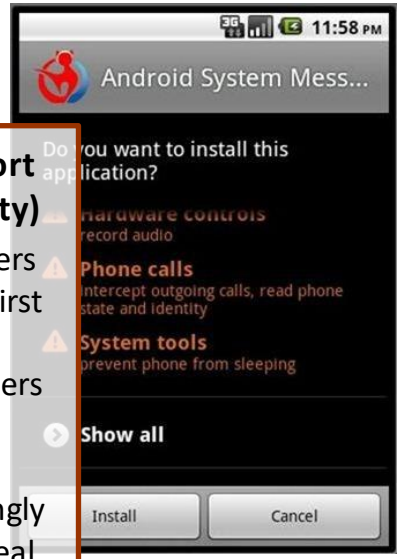
The Ariane Rocket Disaster (1996)

Post Mortem

- Caused due to numeric overflow error
  - Attempt to fit 64-bit format data in 16-bit space
- Cost
  - $100M's for loss of mission
  - Multi-year setback to the Ariane program

Security Vulnerabilities

- Exploits of errors in programs
- Widespread problem
  - Moonlight Maze (1998)
  - Code Red (2001)
  - Titan Rain (2003)
  - Stuxnet (2010)
  - Heartbleed (2014)
- And getting worse …



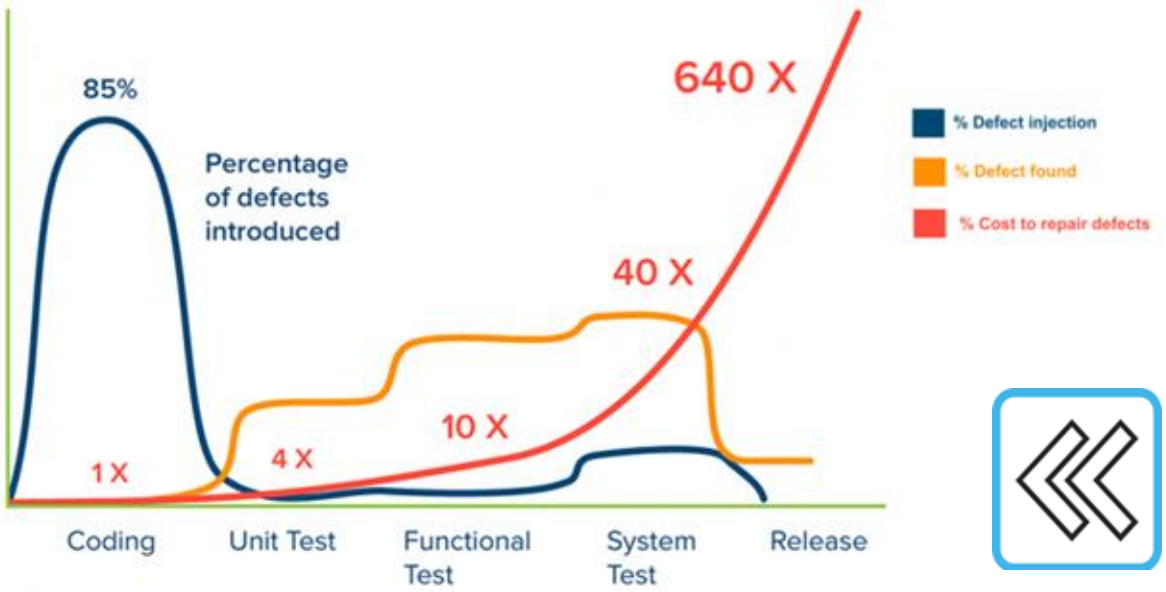
2011 Mobile Threat Report (Lookout™ Mobile Security)

- 0.5-1 million Android users affected by malware in first half of 2011
- 3 out of 10 Android owners likely to face web-based threat each year
- Attackers using increasingly sophisticated ways to steal data and money

READING

From Software Bugs to Security Vulnerabilities

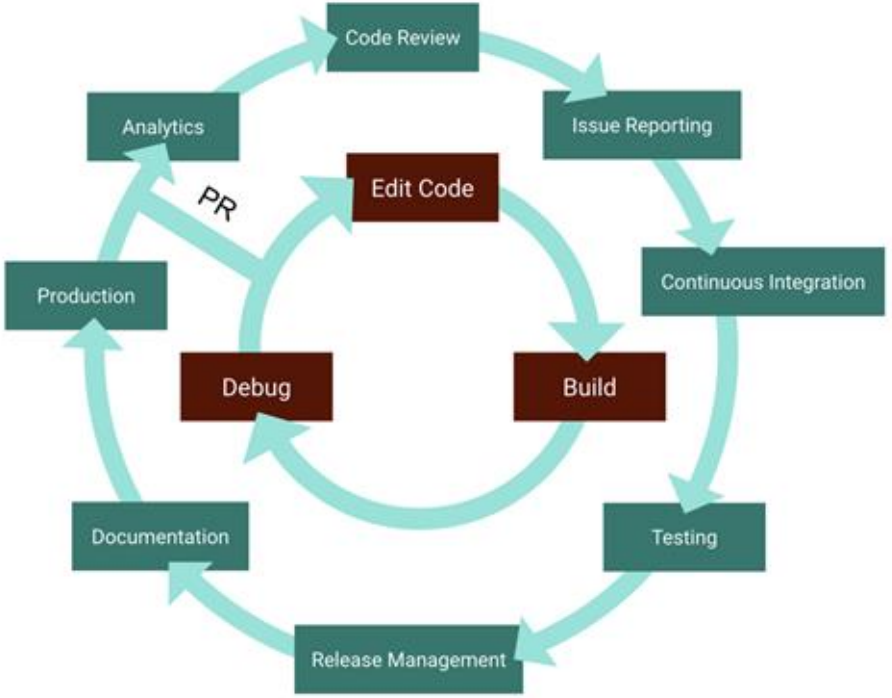
The Cost of Software Errors



Modern Software Development >> Programming

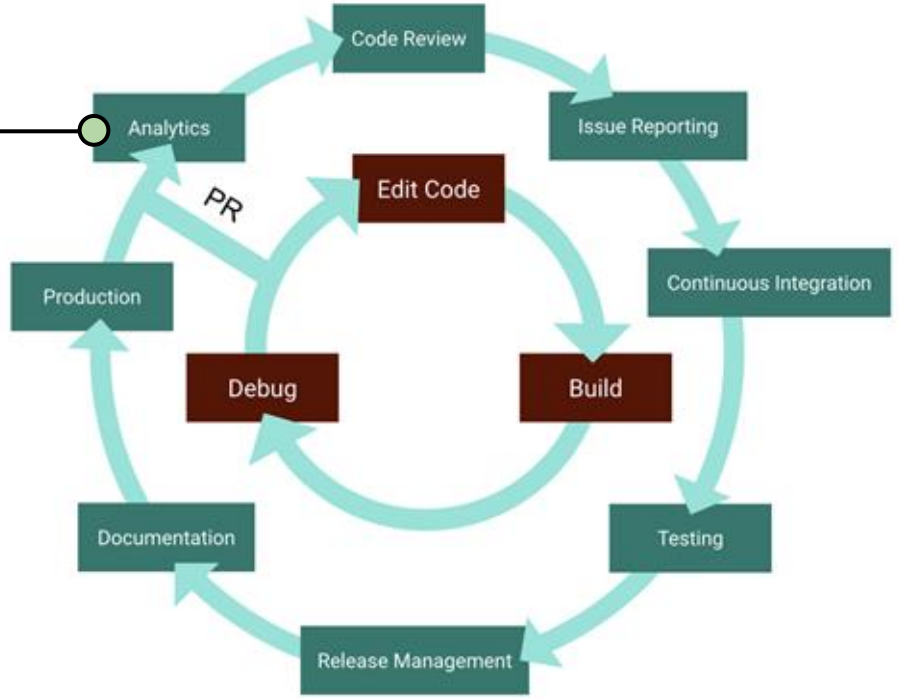
Microsoft

Software Development Life Cycle



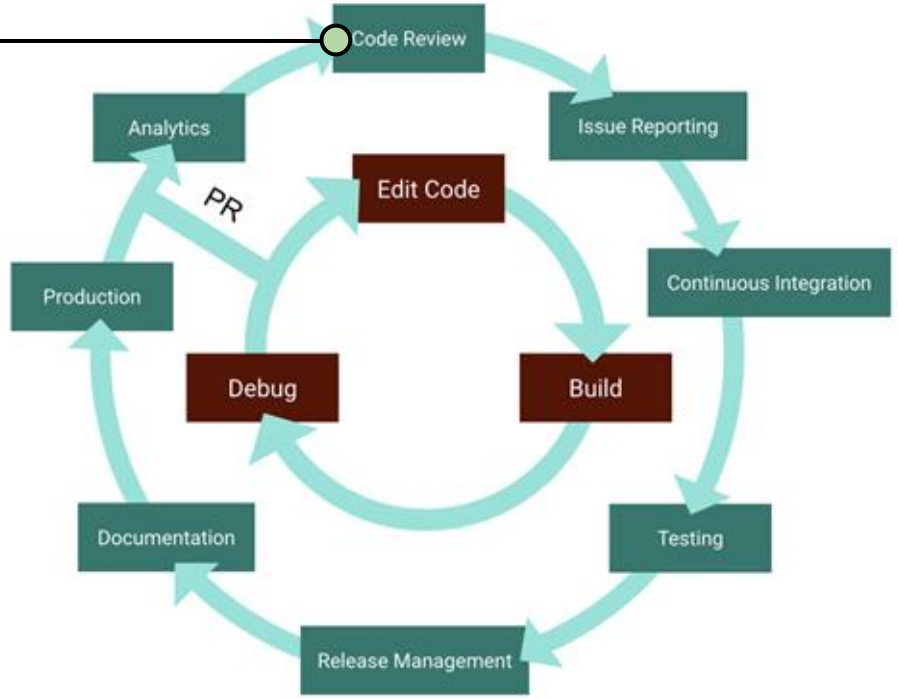
Use-Cases of this Course

How do we systematically find common programming errors in source code before it is deployed?

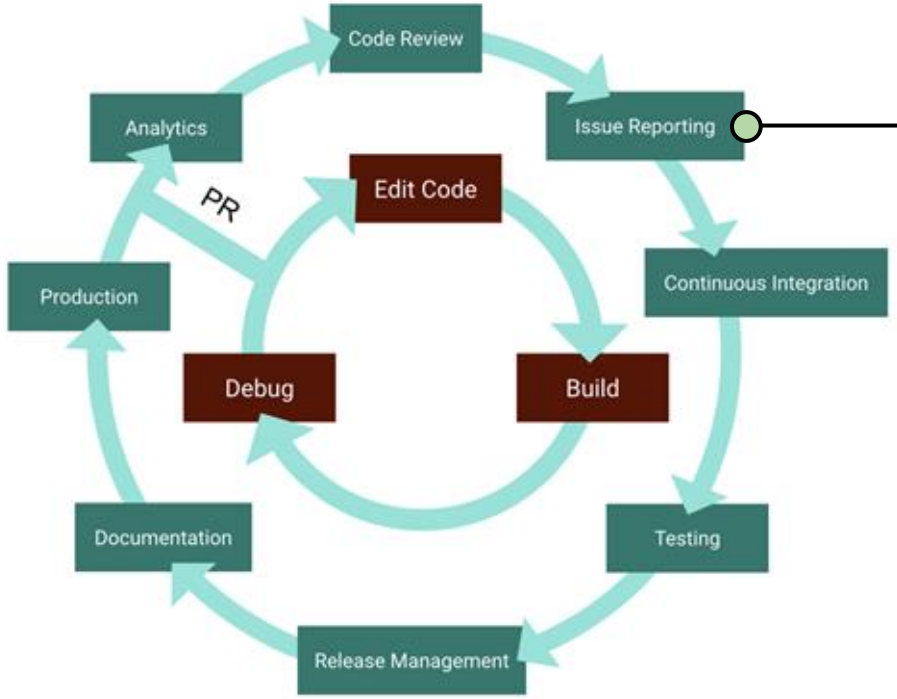


Use-Cases of this Course

How do we specify the intended functionality of each program unit (loop, function, class, etc.)?

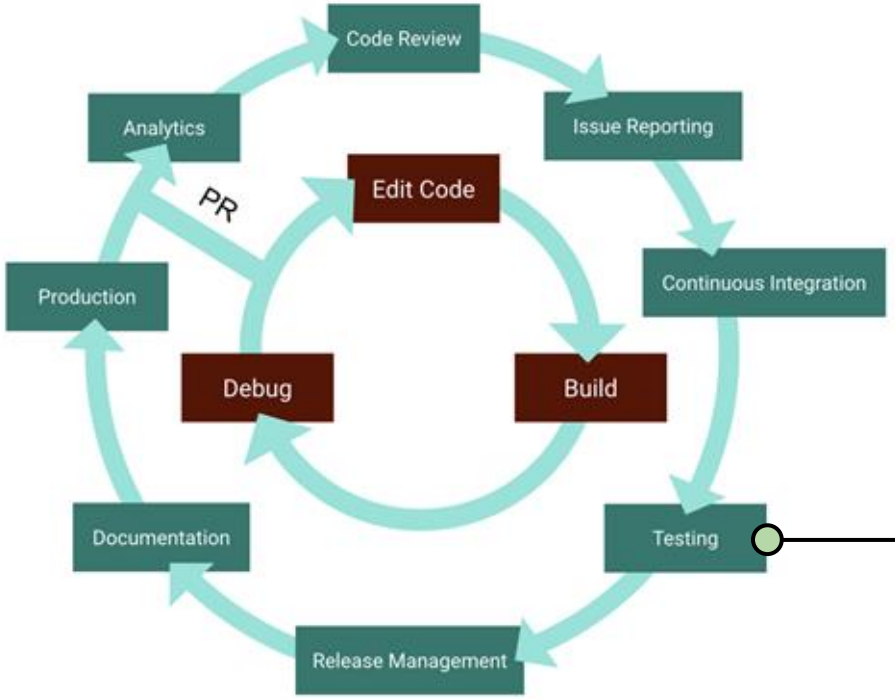


Use-Cases of this Course



How do we minimize a complex crashing test case to isolate the root cause of a bug?

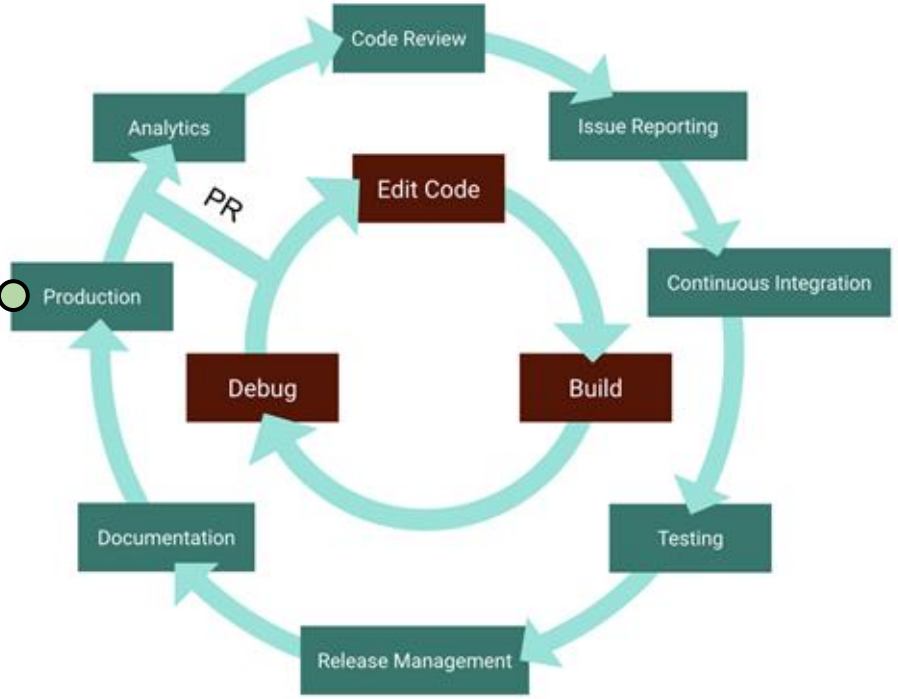
Use-Cases of this Course



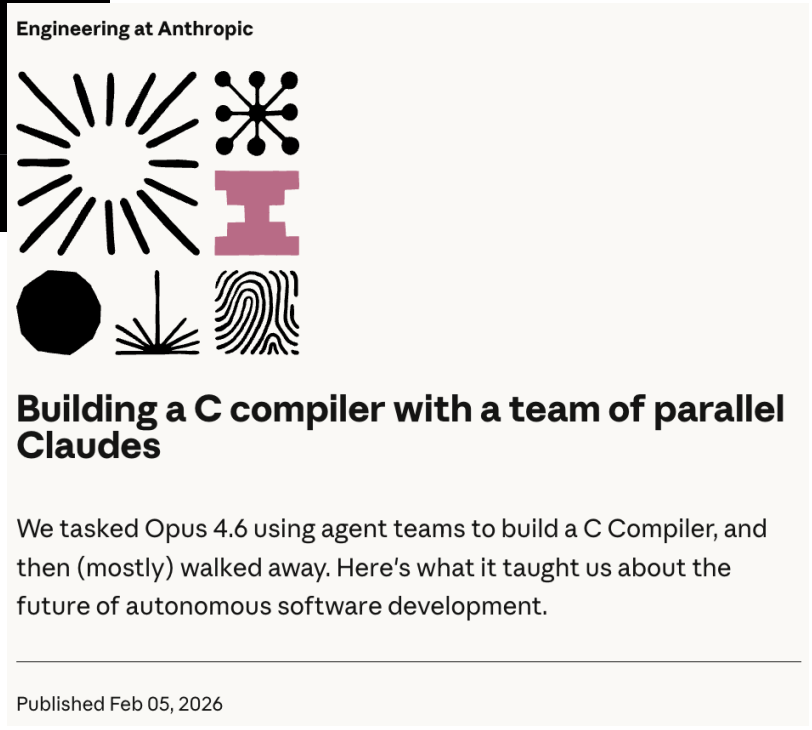
How do we automatically generate tests for libraries, protocols, data structures, and multi-threaded programs?

Use-Cases of this Course

How do we crowdsource the task of discovering the bugs that most affect end-users of a program?

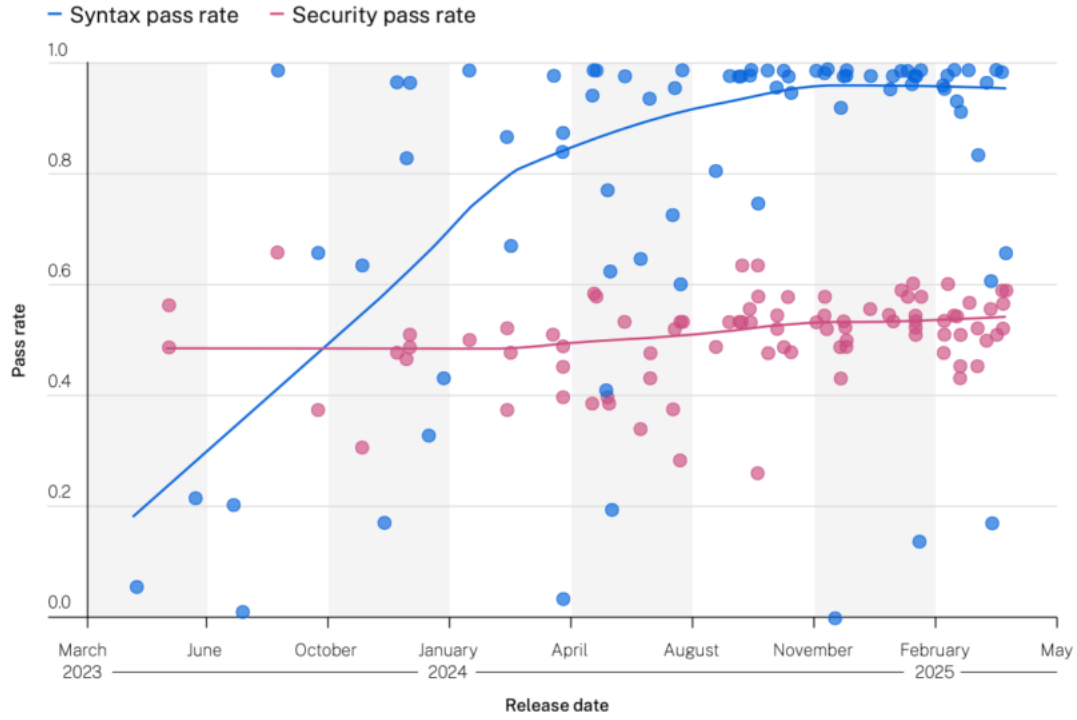


The Elephant in the Room: LLMs and Agents



But the Problem is Far from Solved …

Security and Syntax Pass Rates vs LLM Release Date



Note: Security rate only shown for dates/groups with syntax pass rate > 50%

What's New This Semester

More ambitious labs made possible by LLM agents!

Labs in past semesters

- One prescribed algorithm
- Toy programs, provided tests
- Same labs for everyone

LLM agents

Labs this semester

- LLMs propose, you verify
- Shared benchmarks, real code
- Advanced tier: pick four

Lab 1 Recitation

Lab 1 Resources

- Setup Instructions for the Course VM
  https://cis547.github.io/resources/course-vm/

⚠ Please hold off installing the course VM until we announce on Ed!

- Lab 1: Introduction to Software Analysis
  https://cis547.github.io/labs/lab1/

Introduction to Software Analysis

What is Program Analysis?

- Body of work to discover useful facts about programs
- Broadly classified into three kinds:
  - Dynamic (execution-time)
  - Static (compile-time)
  - Hybrid (combines dynamic and static)

Dynamic Program Analysis

- Infer facts of the program by monitoring its runs
- Examples:

Array bound checking
Purify

Memory leak detection
Valgrind

Datarace detection
Eraser

Finding likely invariants
Daikon

Static Analysis

- Infer facts of the program by inspecting its source (or binary) code
- Examples:

Suspicious error patterns
Lint, FindBugs, Coverity

Checking API usage rules
Microsoft SLAM

Memory leak detection
Facebook Infer

Verifying invariants
ESC/Java

Program Invariants

An invariant at the end of the program is (z == *c*), for some constant *c*. What is *c*?

```
int p(int x) { return x * x; }

void main() {
  int z;
  if (getc() == 'a')
    z = p(6) + 6;
  else
    z = p(-7) - 7;

                          z = ?
}
```

Program Invariants

An invariant at the end of the program is (z == *c*), for some constant *c*. What is *c*?

Disaster averted!

```
int p(int x) { return x * x; }

void main() {
  int z;
  if (getc() == 'a')
    z = p(6) + 6;
  else
    z = p(-7) - 7;
  if (z != 42)
    disaster();
}
```

z = 42

Discovering Invariants By Dynamic Analysis

(z == 42) *might be* an invariant

(z == 30) is *definitely not* an invariant

```
int p(int x) { return x * x; }

void main() {
  int z;
  if (getc() == 'a')
    z = p(6) + 6;
  else
    z = p(-7) - 7;
  if (z != 42)
    disaster();
}
```

z = 42

Discovering Invariants By Static Analysis

(z == 42) *is definitely* ~~*might be*~~ an invariant

(z == 30) is *definitely not* an invariant

```
int p(int x) { return x * x; }

void main() {
  int z;
  if (getc() == 'a')
    z = p(6) + 6;
  else
    z = p(-7) - 7;
  if (z != 42)
    disaster();
}
```

z = 42

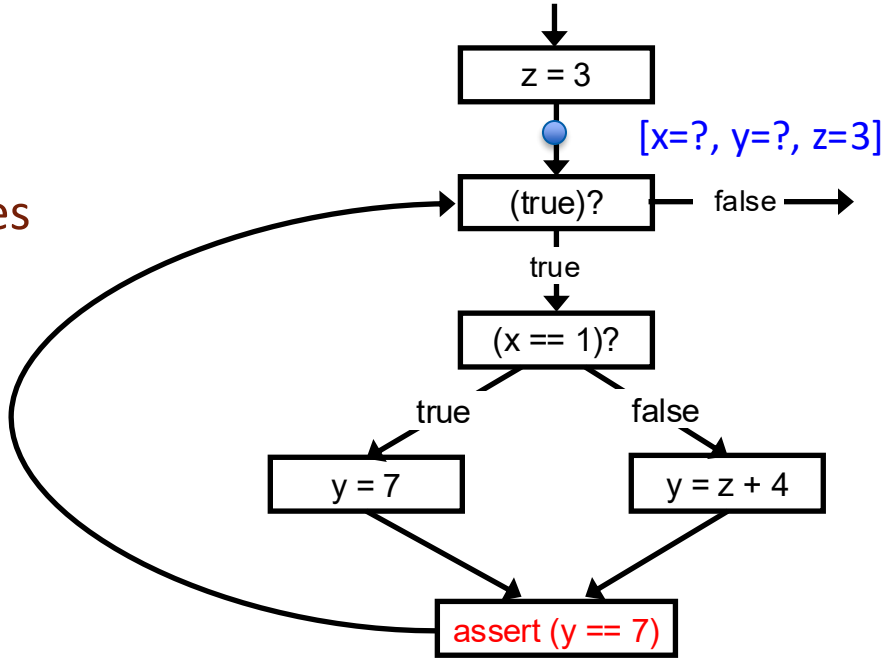
Example Static Analysis Problem

Find variables that have a constant value at a given program point

```
void main() {
  z = 3;
  while (true) {
    if (x == 1)
      y = 7;
    else
      y = z + 4;
    assert(y == 7);
  }
}
```

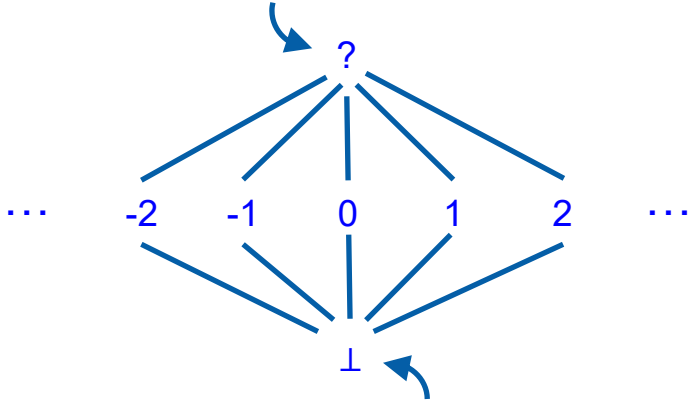
Terminology

- Control-flow graph
- Abstract vs. concrete states
- Termination
- Completeness
- Soundness



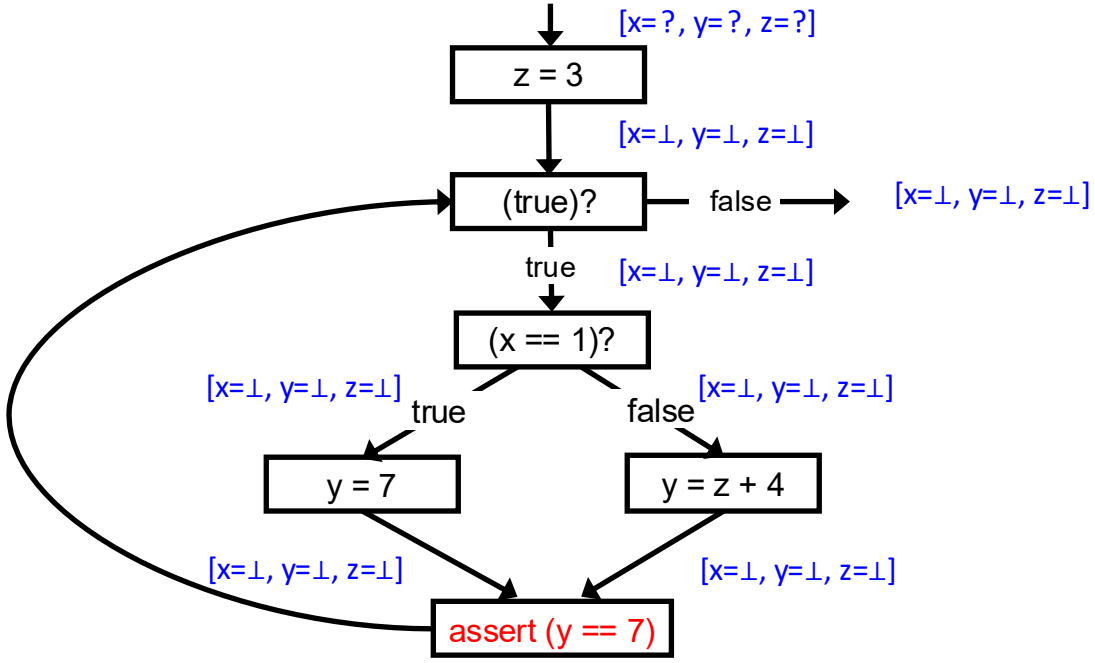
Example Abstract Domain

Value is **unknown** to the analysis

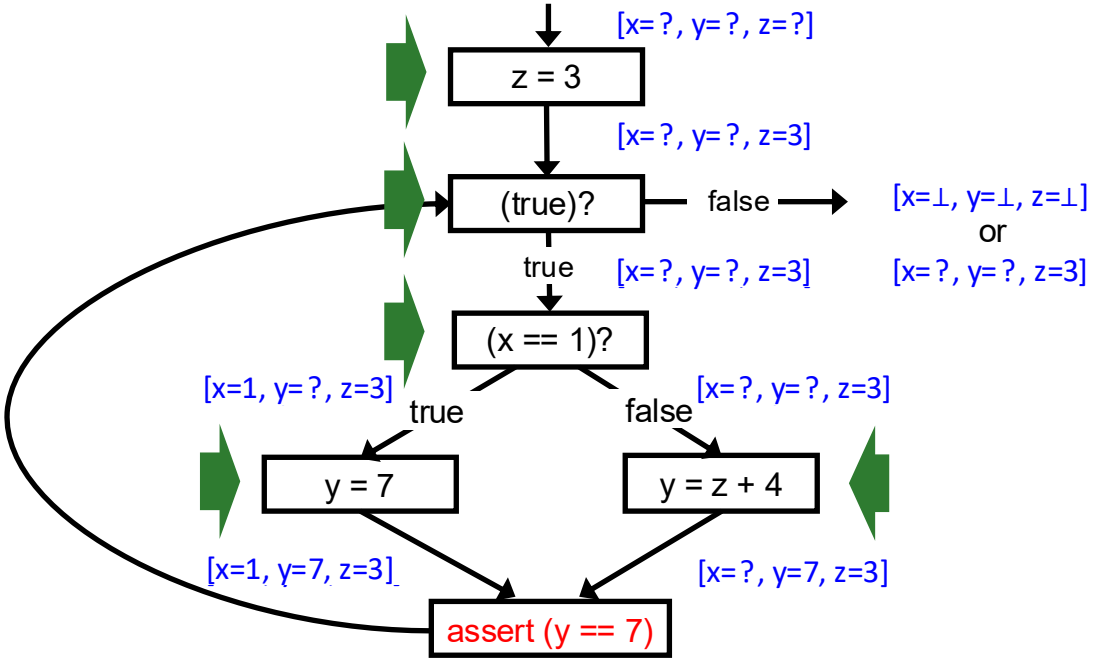


Value is **undefined** by the analysis

Iterative Approximation



Iterative Approximation

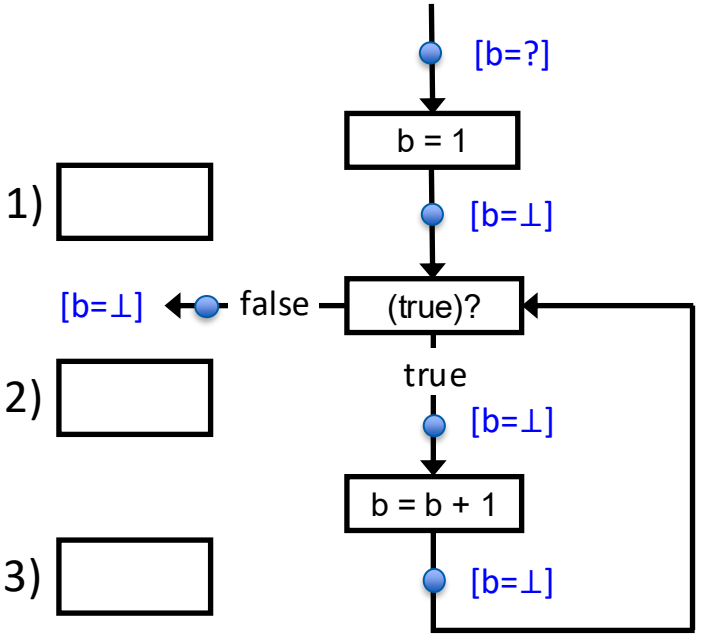


Another Example: Iterative Approximation

Fill in the value of variable b that the analysis infers at:

1) the loop header
2) entry of loop body
3) exit of loop body

Enter “?” if a definite value cannot be inferred.

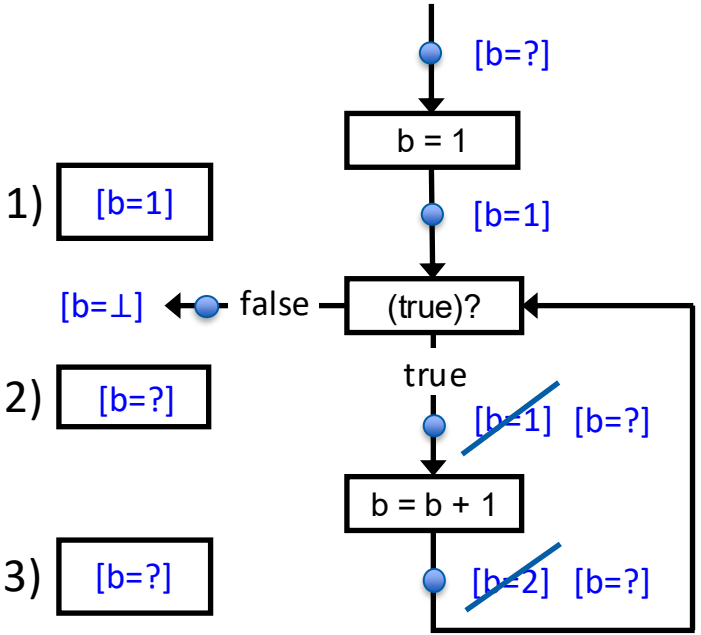


Another Example: Iterative Approximation

Fill in the value of variable b that the analysis infers at:

1) the loop header
2) entry of loop body
3) exit of loop body

Enter “?” if a definite value cannot be inferred.



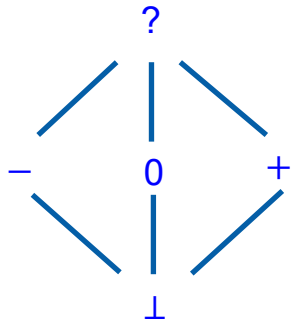
Recap: Parts of a Static Analysis

- Program representation
  - e.g. control-flow graph, AST, or bytecode
- Abstract domain
  - e.g. how to approximate program values
- Transfer functions
  - e.g. assignments, conditionals, merge points, …
- Fixed-point computation algorithm
  - e.g. iterative approximation

QUIZ: Example Static Analysis Problem

Find statements that divide-by-zero

Use abstract domain:

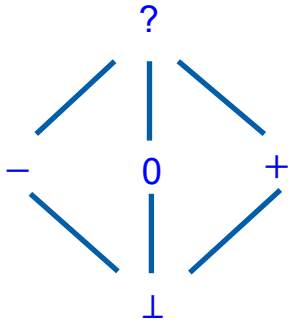


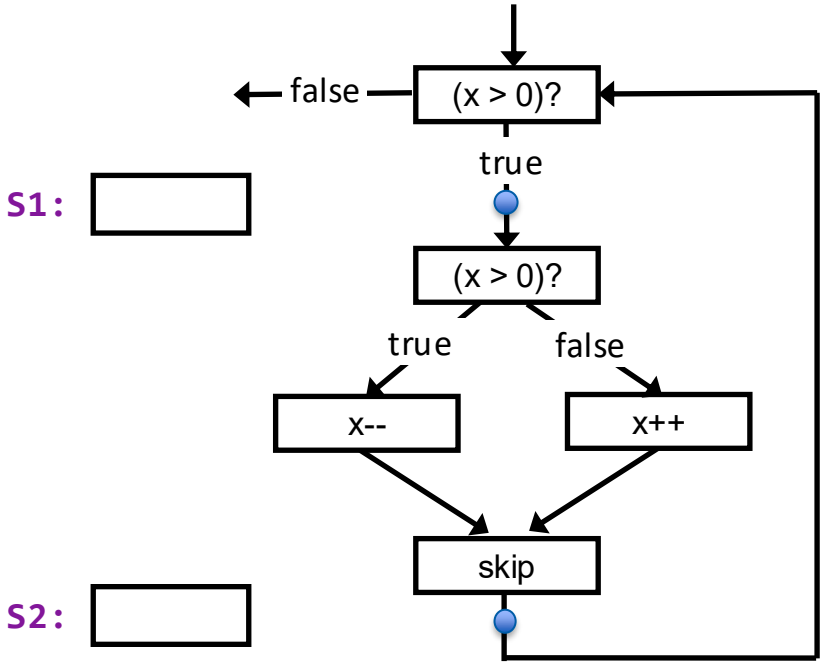

```
void main() {
   while (x > 0) {
S1:    ... 1/x ...
       if (x > 0) {
          x--;
       } else {
          x++;
       }
S2:    ... 1/x ...
   }
}
```

QUIZ: Example Static Analysis Problem

Find statements that divide-by-zero

Use abstract domain:

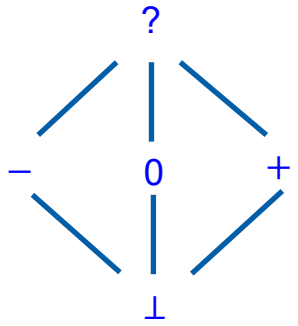


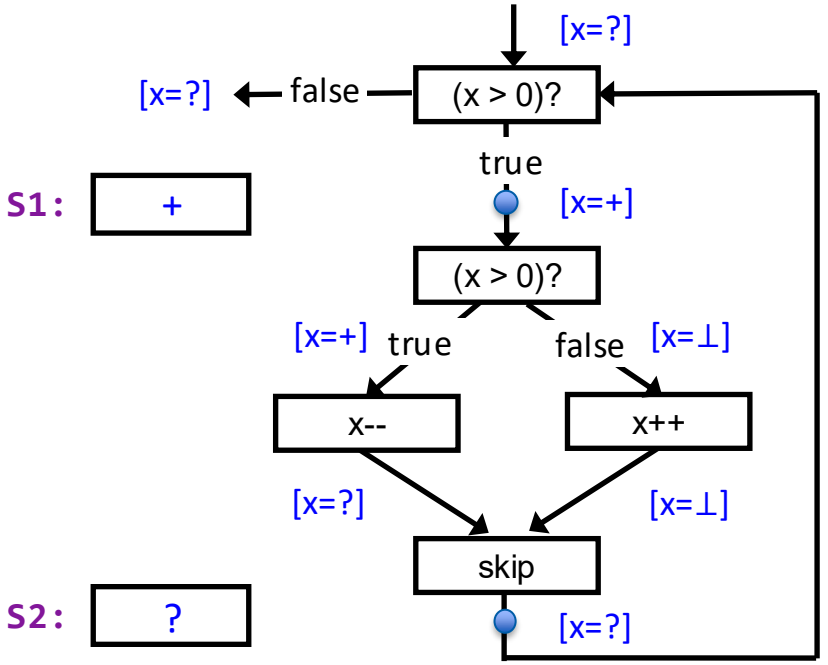


QUIZ: Example Static Analysis Problem

Find statements that divide-by-zero

Use abstract domain:





READING

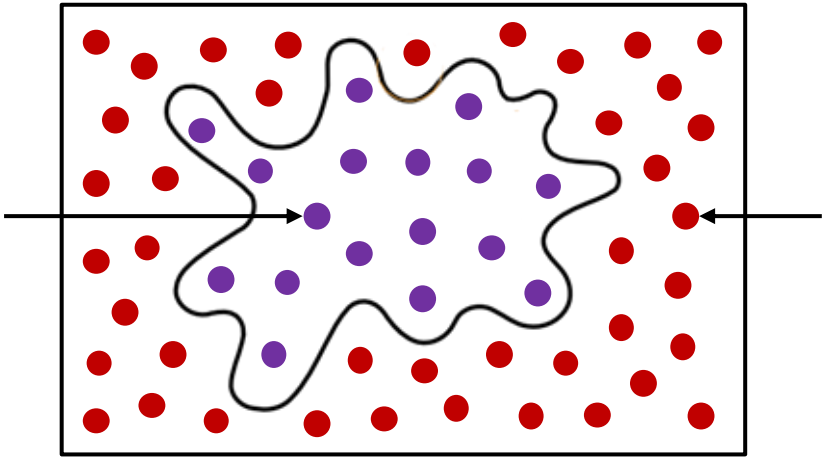
A Menagerie of Program Abstractions

Tradeoffs in Software Analysis

Characterizing Program Analyses

good program

Satisfies property φ



bad program

Violates property φ

Example property φ: no divide-by-zero errors

A Problem: Finding Divide-by-Zero Errors

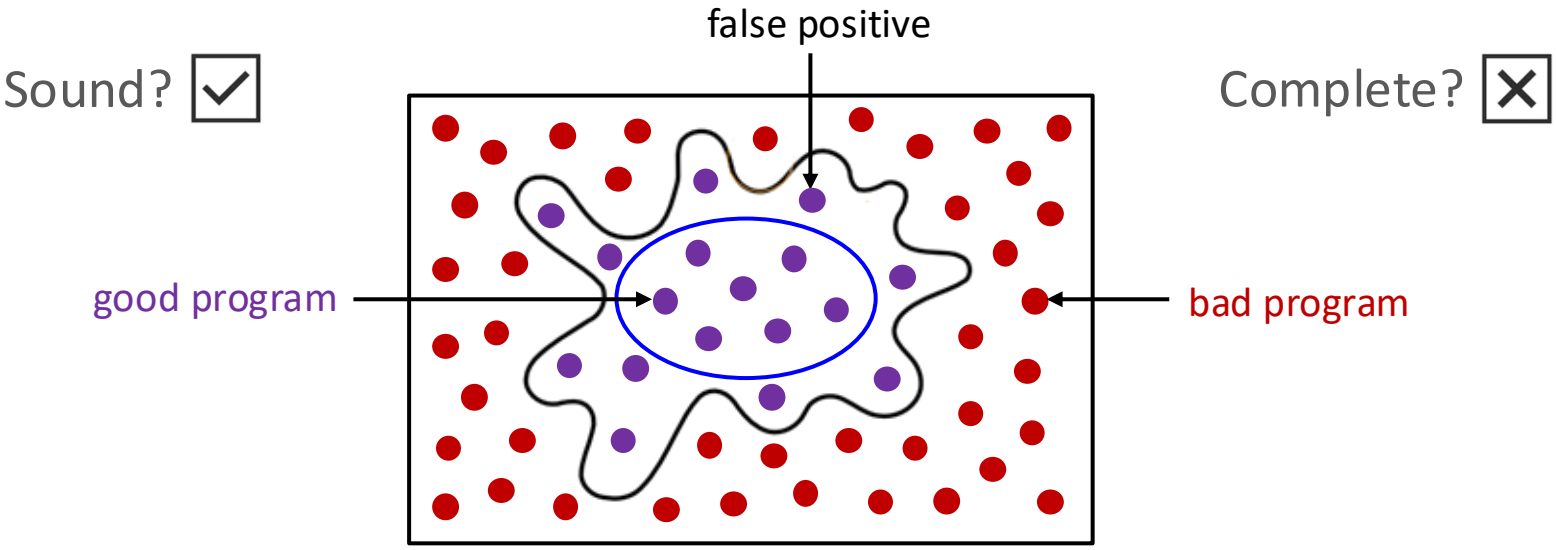
```
void main() {
   while (x > 0) {
      assert(x != 0);
      ... 1/x ...
      if (x > 0)
         x--;
      else
         x++;
   }
}
```

good program (safe, bug-free, …)

```
void main() {
   while (x > 0) {
      if (x > 0)
         x--;
      else
         x++;
      assert(x != 0);
      ... 1/x ...
   }
}
```

bad program (unsafe, buggy, …)

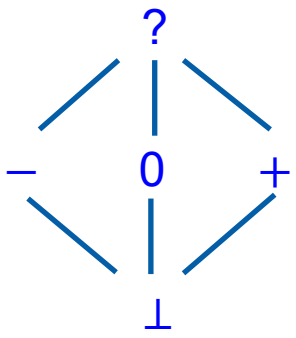
Program Analysis: Kind 1 of 4



Example: A Static Analysis

```
void main() {
   while (x > 0) {
      assert(x != 0);
      ... 1/x ...
      if (x > 0)
         x--;
      else
         x++;
   }
}
```

[x = ?]
or
[x = +]



good program

```
void main() {
   while (x > 0) {
      if (x > 0)
         x--;
      else
         x++;
      assert(x != 0);
      ... 1/x ...
   }
}
```

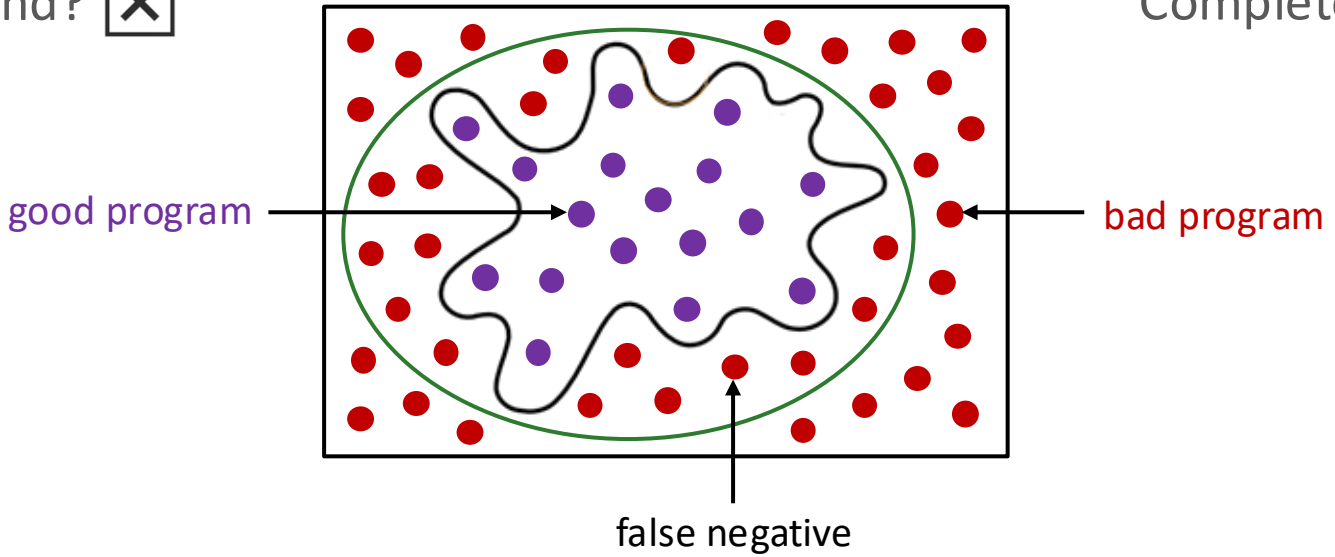
[x = ?]

bad program

Program Analysis: Kind 2 of 4

Sound? ☒

Complete? ☑



Example: A Dynamic Analysis

```
void main() {
   while (x > 0) {
      assert(x != 0);
      ... 1/x ...
      if (x > 0)
         x--;
      else
         x++;
   }
}
```

[x = 3] ←

[x = -3] →

```
void main() {
   while (x > 0) {
      if (x > 0)
         x--;
      else
         x++;
      assert(x != 0);
      ... 1/x ...
   }
}
```

good program

bad program

QUIZ: Dynamic vs. Static Analysis

Match each box with its corresponding feature.

	Dynamic	Static
Cost		
Effectiveness		

A. Unsound (may miss errors)

B. Proportional to program's run time

C. Proportional to program's size

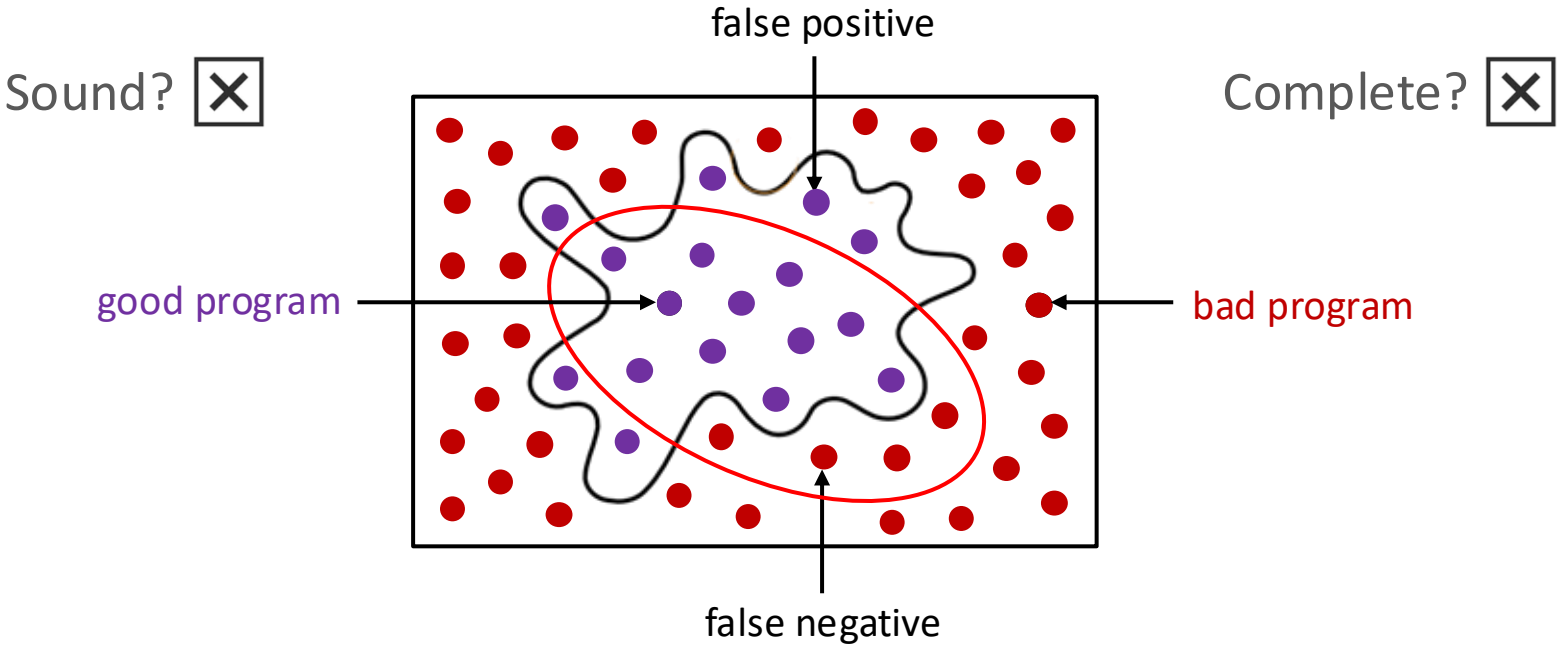
D. Incomplete (may report spurious errors)

QUIZ: Dynamic vs. Static Analysis

Match each box with its corresponding feature.

	Dynamic	Static
Cost	B. Proportional to program's run time	C. Proportional to program's size
Effectiveness	A. Unsound (may miss errors)	D. Incomplete (may report spurious errors)

Program Analysis: Kind 3 of 4



Precision and Recall

		Analysis Outcome	
		Accept	Reject
Program's Ground Truth	Good	True Negative	False Positive
	Bad	False Negative	True Positive

$$precision = \frac{\text{\# True Positives}}{\text{\# Rejected}}$$

$$recall = \frac{\text{\# True Positives}}{\text{\# Bad}}$$

F-Measure

$$precision = \frac{\text{\# True Positives}}{\text{\# Rejected}} \qquad recall = \frac{\text{\# True Positives}}{\text{\# Bad}}$$

$$\text{F measure} = \frac{2}{\frac{1}{precision} + \frac{1}{recall}}$$

- Caveat: May want to weigh precision and recall differently in practice
- Determined by the application's demands

QUIZ: Comparing Program Analyses

Calculate the precision, recall, and F-measure of each of the following analyses:

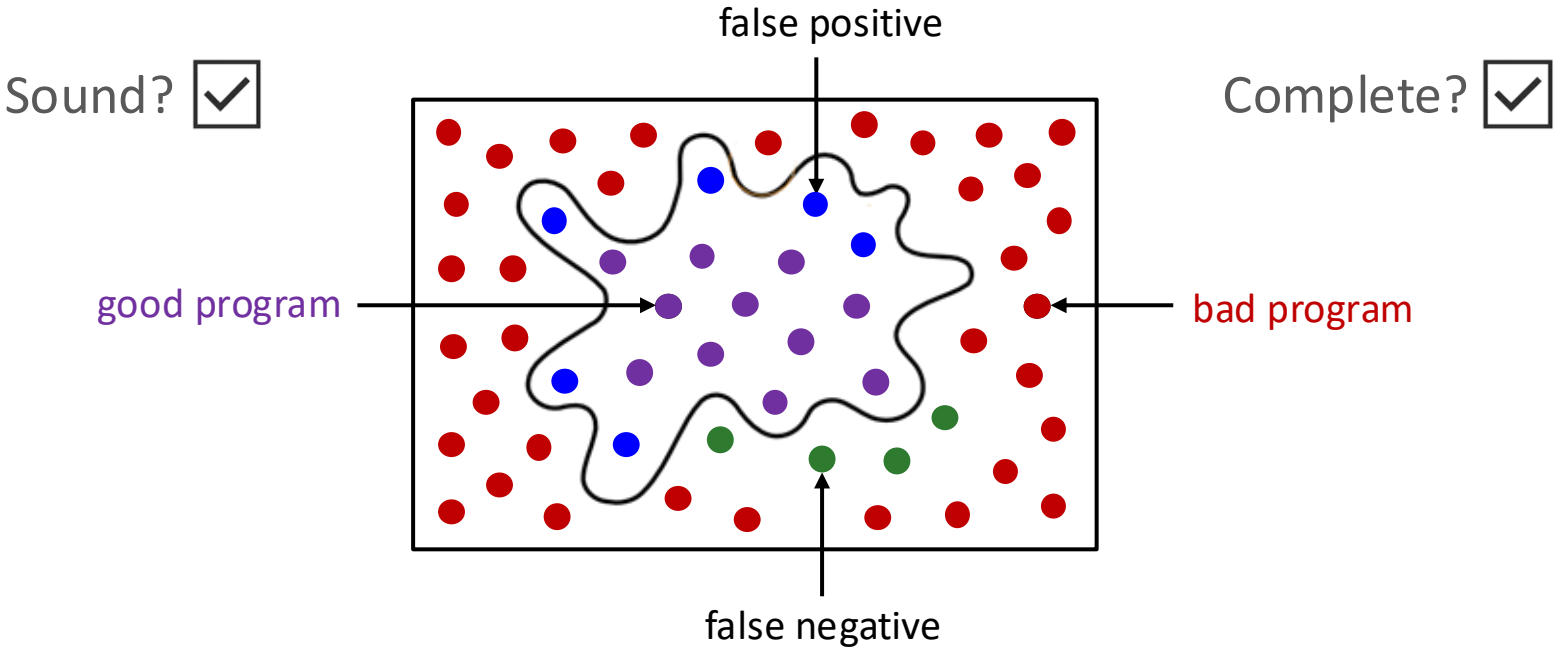
	precision	recall	F-measure
sound and complete analysis			
sound analysis with 40% false positive (FP) rate			
complete analysis with 40% false negative (FN) rate			
analysis with 30% FP rate and 70% FN rate			

QUIZ: Comparing Program Analyses

Calculate the precision, recall, and F-measure of each of the following analyses:

	precision	recall	F-measure
sound and complete analysis	1.00	1.00	1.00
sound analysis with 40% false positive (FP) rate	0.60	1.00	0.75
complete analysis with 40% false negative (FN) rate	1.00	0.60	0.75
analysis with 30% FP rate and 70% FN rate	0.70	0.30	0.42

Program Analysis: Kind 4 of 4



Undecidability of Program Properties

- Can program analysis be sound and complete?
  - Not if we want it to terminate!
- Questions like “is a program point reachable on some input?” are undecidable
- Designing a program analysis is an art
  - Tradeoffs dictated by consumer

READING

Undecidability of Program Properties

Who Needs Program Analysis?

Three primary consumers of program analysis:

- Compilers
- Software Quality Tools
- Integrated Development Environments (IDEs)

Compilers

- Bridge between high-level languages and architectures
- Use program analysis to generate efficient code

```
int p(int x) { return x*x; }

void main(int arg) {
   int z;
   if (arg != 0)
      z = p(6) + 6;
   else
      z = p(-7) - 7;

   print(z);
}
```

z = 42

```
int p(int x) { return x*x; }

void main() {
   print(42);
}
```

- Runs faster
- More energy-efficient
- Smaller in size

Software Quality Tools

- Primary focus of this course
- Tools for testing, debugging, and verification
- Use program analysis for:
  - Finding programming errors
  - Proving program invariants
  - Generating test cases
  - Localizing causes of errors
  - …

```
int p(int x) { return x * x; }

void main() {
  int z;
  if (getc() == 'a')
    z = p(6) + 6;
  else
    z = p(-7) - 7;
  if (z != 42)
    disaster();
}
```

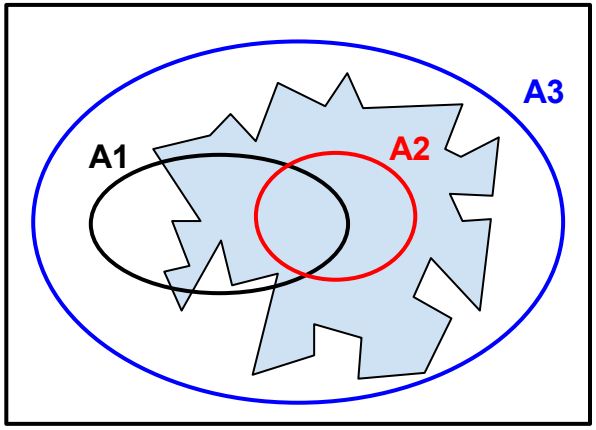
z = 42

Integrated Development Environments

- Examples: Eclipse and Microsoft Visual Studio
- Use program analysis to help programmers:
  - Understand programs
  - Refactor programs
    - Restructuring a program without changing its behavior
- Useful in dealing with large, complex programs

QUIZ (1/3): Choosing a Program Analysis

For each analysis (**A1**, **A2**, **A3**), state whether it is sound, complete, both, or neither.

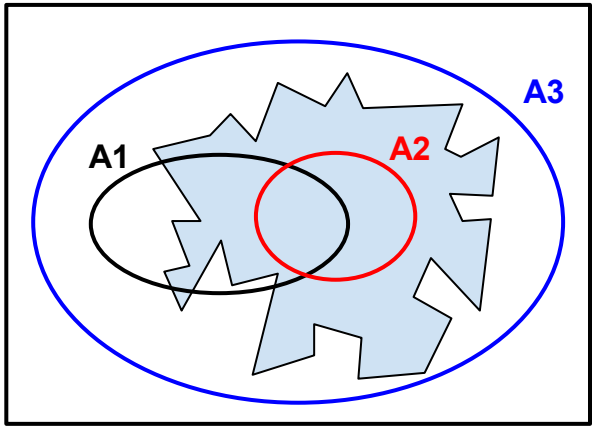


space of all programs

	Sound?	Complete?
A1		
A2		
A3		

QUIZ (1/3): Choosing a Program Analysis

For each analysis (**A1**, **A2**, **A3**), state whether it is sound, complete, both, or neither.



space of all programs

	Sound?	Complete?
A1	No	No
A2	Yes	No
A3	No	Yes

QUIZ (2/3): Choosing a Program Analysis

NASA engineer Bob has been asked to apply program analysis to check divide-by-zero errors in software that will control NASA's next billion-dollar space mission.

	Sound?	Complete?
A1	No	No
A2	Yes	No
A3	No	Yes

Which analysis (**A1**/A2/A3) should Bob use?

QUIZ (2/3): Choosing a Program Analysis

NASA engineer Bob has been asked to apply program analysis to check divide-by-zero errors in software that will control NASA's next billion-dollar space mission.

	Sound?	Complete?
A1	No	No
A2	Yes	No
A3	No	Yes

Which analysis (**A1**/A2/A3) should Bob use?

A2

QUIZ (3/3): Choosing a Program Analysis

Microsoft developer Ann has agreed to apply program analysis to check divide-by-zero errors in her programs on the condition that it will not produce false alarms.

	Sound?	Complete?
A1	No	No
A2	Yes	No
A3	No	Yes

Which analysis (**A1**/**A2**/**A3**) should Ann use?

QUIZ (3/3): Choosing a Program Analysis

Microsoft developer Ann has agreed to apply program analysis to check divide-by-zero errors in her programs on the condition that it will not produce false alarms.

	Sound?	Complete?
A1	No	No
A2	Yes	No
A3	No	Yes

Which analysis (**A1**/**A2**/**A3**) should Ann use?

A3

What Have We Learned?

- What is program analysis?
- Dynamic analysis vs. static analysis: pros and cons
- Program invariants
- Iterative approximation method for static analysis
- Characterizing program analyses

  Undecidability => program analysis cannot ensure
  termination + soundness + completeness

- Who needs program analysis?